

EPIC 07 — Task 01: High-Level System Design

System: E-Commerce Order Management System

We will design a **scalable and reliable e-commerce system** where users can browse products, place orders, make payments, and track their orders.

1. Functional Requirements

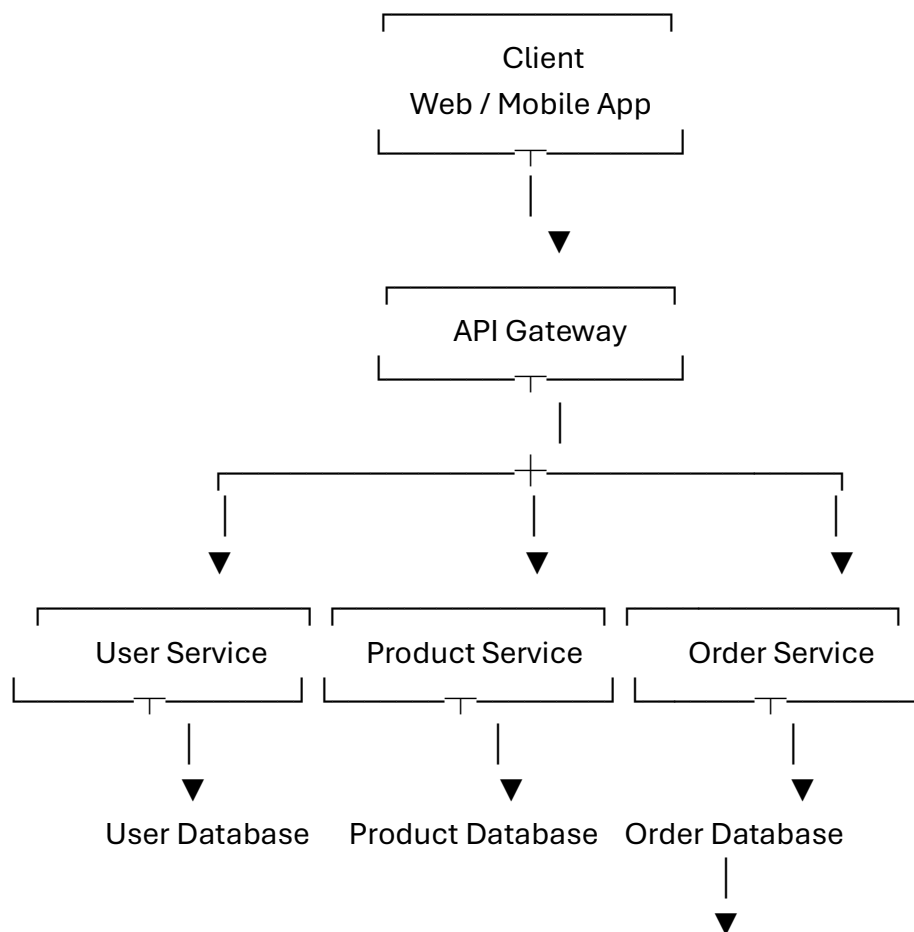
The system should provide:

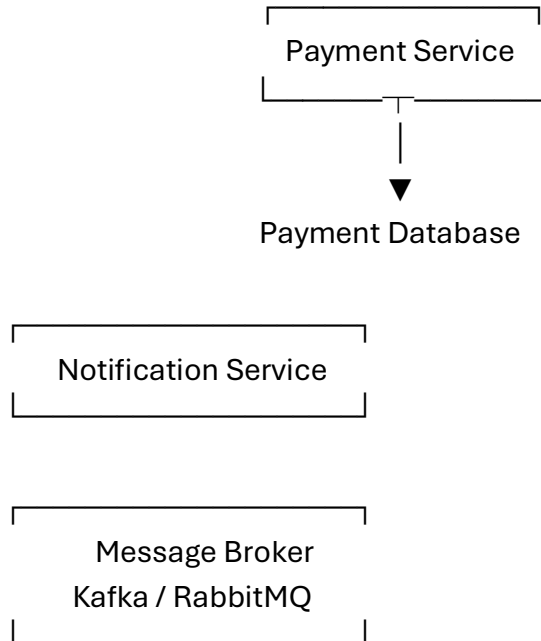
1. **User Management**
 - a. User registration and login.
 - b. User profile management.
 - c. Authentication and authorization.
2. **Product Management**
 - a. Add, update, delete, and view products.
 - b. Search and filter products.
 - c. Check product availability.
3. **Order Management**
 - a. Create an order.
 - b. View the order details.
 - c. Update/cancel an order.
 - d. Track the order status.
4. **Payment Management**
 - a. Process payments.
 - b. Maintain payment status.
 - c. Handle failed payments and refunds.
5. **Notification**
 - a. Send order confirmation.
 - b. Send payment and delivery notifications.

2. Non-Functional Requirements

- **Scalability:** System should support increasing users and orders.
- **Availability:** System should remain available even if one service fails.
- **Reliability:** Orders and payments should not be lost or duplicated.
- **Performance:** APIs should respond quickly, preferably within a few hundred milliseconds for normal operations.
- **Security:** Authentication, authorization, encryption, and secure payment processing.
- **Maintainability:** Services should be independently developed and deployed.
- **Monitoring:** Logs, metrics, health checks, and alerts should be available.

3. High-Level Architecture





4. Main Components

API Gateway

Acts as the single-entry point for clients.

Responsibilities:

- Routing requests.
- Authentication.
- Rate limits.
- Load balancing.
- Request validation.

User Service

Manages:

- Registration.
- Login.
- User profiles.

- Roles and permissions.

Product Service

Manages:

- Product information.
- Categories.
- Prices.
- Inventory availability.

Order Service

Manages:

- Creating orders.
- Order history.
- Order status.
- Cancellation.

Payment Service

Manages:

- Payment processing.
- Payment confirmation.
- Refunds.
- Payment status.

Notification Service

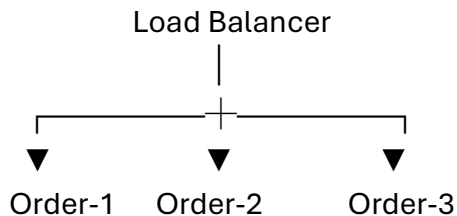
Sends:

- Email notifications.
- SMS/push notifications.
- Order confirmation.
- Delivery updates.

5. Scalability

The system can scale horizontally.

For example:



Instead of increasing the size of one server, we can create multiple instances of a service.

A **load balancer** distributes requests among available instances.

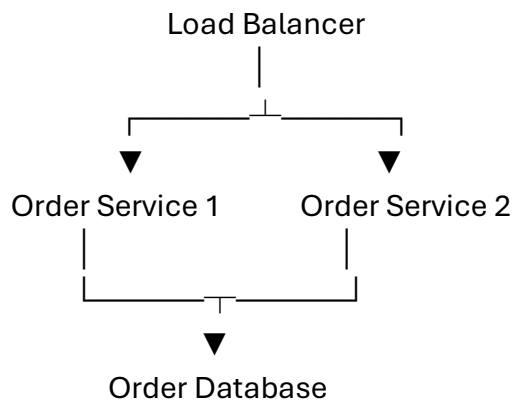
Frequently accessed data can also be stored in a **cache such as Redis** to reduce database load.

6. Availability

To provide high availability:

- Run multiple instances of important services.
- Use load balancing.
- Use database replication.
- Use health checks.
- Automatically restart failed service instances.
- Avoid a single point of failure.

Example:



If one Order Service instance fails, traffic can be routed to the other instance.

7. Reliability

Reliability can be improved using:

- Database transactions.
- Retry mechanisms.
- Idempotent APIs.
- Message queues.
- Database backups.
- Error handling.
- Circuit breakers.

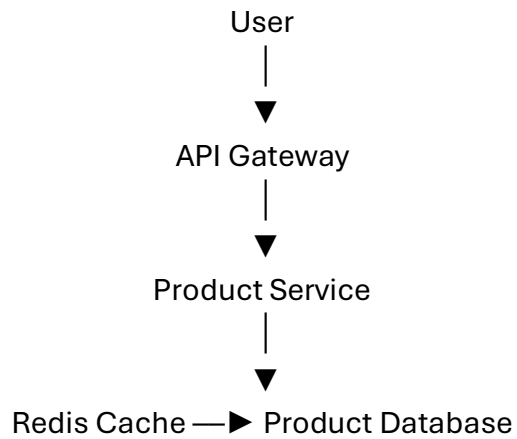
For example, if the notification service is temporarily unavailable, the order should still be created. The notification can be sent later through a message broker.

8. Performance

Performance can be improved by:

- Using Redis caching.
- Database indexing.
- Pagination for large data.
- Asynchronous processing.
- Connection pooling.
- Load balancing.
- Optimized database queries.

Example:



If the product is already in Redis, the system does not need to query the database.

9. Communication

Two types of communication can be used:

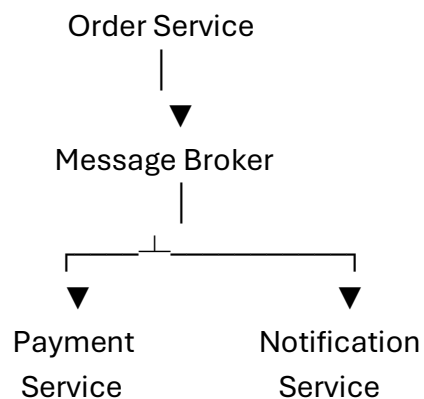
Synchronous

REST APIs can be used when an immediate response is required.

Client → API Gateway → Order Service

Asynchronous

Kafka or RabbitMQ can be used for events.



10. Security

The system should use:

- HTTPS.
- JWT/OAuth2 authentication.
- Role-based authorization.
- Password hashing.
- Input validation.
- API rate limits.
- Secure database credentials.
- Encryption for sensitive data.

11. Database Design

Each microservice can own its database:

User Service → User DB

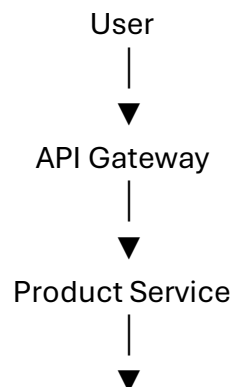
Product Service → Product DB

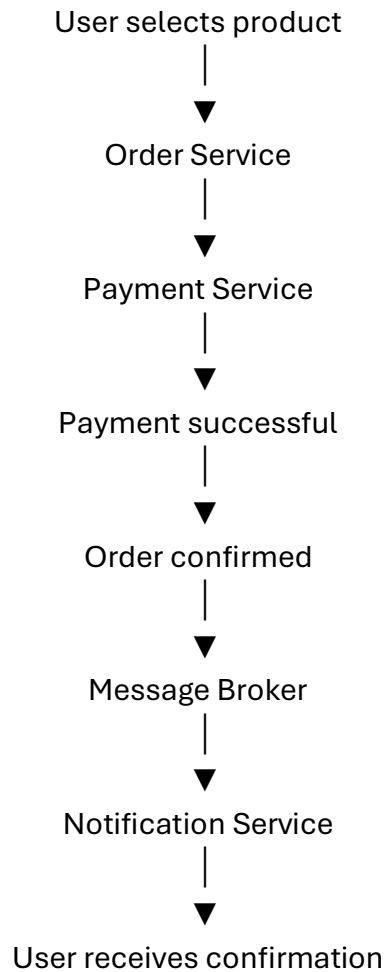
Order Service → Order DB

Payment Service → Payment DB

This keeps services loosely coupled and allows each service to scale independently.

12. Final System Flow





Conclusion

This **E-Commerce Order Management System** satisfies the Task 01 requirements by providing **functional features, scalability, availability, reliability, performance, and security**.

The microservices architecture allows each service to be developed, deployed, and scaled independently.